Assignment 1: Analysis of SVD for Fine-Tuning LLMs

蕭至恆 (Student ID 413551034)

GitHub: https://github.com/CH-Joshua-Hsiao/NYCU-BigDataAnalytics/tree/main/HW1

Setup: All runs used one RTX 3090 (24 GB) on my own computer, with micro-batch 4 and 32 accumulation steps. The SVD step ran on the CPU.

Question 1: Accuracy of the original and fine-tuned models

Accuracy	Original LLM	Fine-Tuned (RANK = 16)	Fine-Tuned (RANK = 64)	Fine-Tuned (RANK = 256)
GSM8K	3.4%	15.6%	20.7%	27.3%
MATH	0.1%	3.7%	4.8%	5.4%

- **Original model.** The original model is very weak: 3.4% on GSM8K and 0.1% on MATH. It is a base model, not a chat model, so it does not really solve the question. In about half of the answers, it just repeats the same sentence again and again. Only 37 GSM8K answers and 41 MATH answers give a clear final answer (like "The answer is:", "####" or \boxed{}). For MATH, 959 answers have no answer that the parser can find. After fine-tuning on MetaMathQA, the model learns to solve step by step and to write the answer in the right format. So even rank 16 goes up to 15.6% / 3.7%.
- **Effect of rank.** When the rank is bigger, the accuracy is higher on both datasets: GSM8K 15.6% → 20.7% → 27.3%, and MATH 3.7% → 4.8% → 5.4%. A bigger rank means more trainable parameters (0.9% → 3.6% → 14.6% of the model). Also, in PiSSA, a bigger rank means the adapter starts from more of the main (principal) parts of the weight. So with the same learning rate and one epoch, the model can change more. The final training loss also goes down (0.455 → 0.414 → 0.375).
- **Format or reasoning?** I also checked if the model only learns the answer format. The table below shows how many answers have a clear final answer, and the accuracy of these answers. On GSM8K, the number is almost the same after rank 16 (951, 957, 954), so the format is already learned. But the accuracy still goes up (16.4% → 21.6% → 28.6%). This means a bigger rank really helps the reasoning, not only the format. On MATH, both become better: more clear answers (452 → 547) and higher accuracy (8.2% → 9.9%). However, the reasoning is still not good. Many answers have a nice format but wrong calculation.

Clear final answers (accuracy of them)	Original LLM	Fine-Tuned (RANK = 16)	Fine-Tuned (RANK = 64)	Fine-Tuned (RANK = 256)
GSM8K	37 (13.5%)	951 (16.4%)	957 (21.6%)	954 (28.6%)
MATH	41 (2.4%)	452 (8.2%)	523 (9.2%)	547 (9.9%)

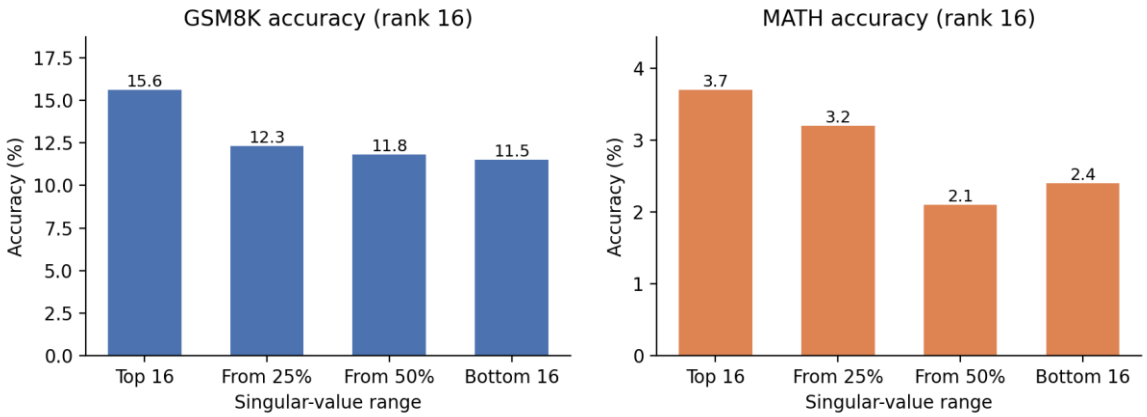
- **GSM8K vs. MATH.** GSM8K improves much more than MATH (+11.7 points vs. +1.7 points from rank 16 to 256). GSM8K questions are simple word problems, and they are similar to the MetaMathQA training data. Its accuracy is still going up at rank 256. MATH questions are much harder. They need more math knowledge and answers in LaTeX form (fractions, square roots, intervals). A 1B model cannot learn this from 25k samples in one epoch. About 45% of MATH answers still have no answer that the parser can find, so MATH improves only a little.

Question 2: Training time

	Fine-Tuned (RANK = 16)	Fine-Tuned (RANK = 64)	Fine-Tuned (RANK = 256)
Training Time (min.)	36.0	38.8	64.0
Trainable Parameters	11.3 M	45.1 M	180.4 M

- **Trend.** From rank 16 to 64, the trainable parameters become 4 times more, but the training time only increases by 2.7 minutes (+8%). From rank 16 to 256, the parameters become 16 times more, and the time becomes 64.0 minutes (+78%). So the training time increases with the rank, but much slower than the number of parameters.
- **Reason.** Most of the training time is used to run the big frozen model (1.2B parameters) forward and backward. This part does not depend on the rank. The adapter only adds two small matrix multiplications. Their cost is r(d+k), while the original weight costs dk. For a 2048×2048 layer, this is only about 1.6% more at r = 16, but about 25% more at r = 256. The adapter also needs its own weight gradients. In addition, rank 256 has 180M trainable parameters, and AdamW keeps extra states for each of them, so the optimizer step is also slower. That is why the time is almost the same for small ranks, and only becomes clearly longer at rank 256. The SVD step at the beginning (about 100 seconds on the CPU) is not included. It takes the same time for all ranks, because the code always computes the full SVD.

Question 3: Fine-tuning different singular-value ranges (rank 16)

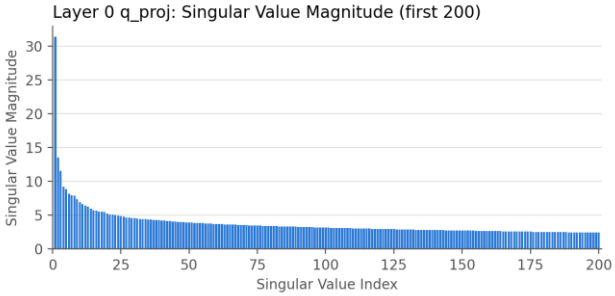


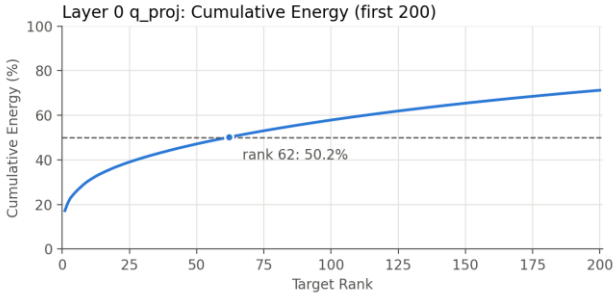
Range	Top 16	From 25%	From 50%	Bottom 16
GSM8K	15.6%	12.3%	11.8%	11.5%
MATH	3.7%	3.2%	2.1%	2.4%

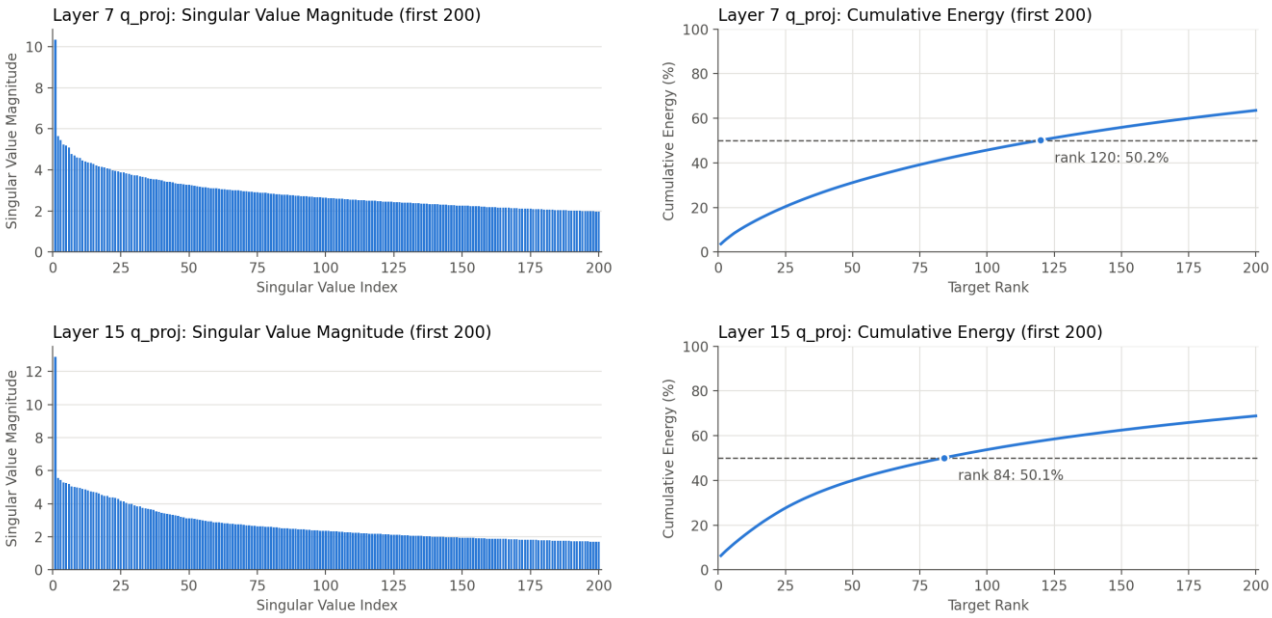
- **Trend.** The top 16 singular values give the best result on both datasets (15.6% on GSM8K, 3.7% on MATH). When we use smaller singular values, the accuracy becomes lower: 12.3% / 11.8% / 11.5% on GSM8K and 3.2% / 2.1% / 2.4% on MATH (from 25%, from 50%, bottom). The training loss has the same order (average of the last 10 steps: 0.419, 0.462, 0.467, 0.476). The three non-top ranges are very close, within about 1 point. With only 1,000 test questions, the random error is about 1 point on GSM8K and 0.5 point on MATH. So I think these three are about the same, and only the top 16 is clearly better. The top 16 is better in both format and reasoning: on GSM8K it gives more clear answers (951 vs. 859-904), and these answers are more accurate (16.4% vs. 12.8-13.5%).
- **Why the top 16 is the best.** In PiSSA, the adapter starts as A = U_r√S_r and B = √S_r V_r^T. So the model can only change the part of the weight that we choose. The top singular values have most of the energy. From Q4, the top 16 have 15-35% of the energy of q_proj, but 16 values at the 25% or 50% position have only about 0.1-0.7%. These top directions are the ones the layer really uses, so changing them has the biggest effect. Also, A and B are bigger at the start, so their gradients are bigger ($\partial L/\partial A \propto B$, $\partial L/\partial B \propto A$), and the model learns faster. The middle and bottom directions are very small and the original model almost does not use them, so changing them helps less.
- **Why the other ranges still work.** Even the bottom 16 (singular values close to 0) is much better than the original model (3.4% → 11.5% on GSM8K). One reason is AdamW: it scales each update by the gradient size, so even very small A and B still move about one learning rate per step. Also, 16 dimensions are enough to learn the answer format and some simple reasoning. Using the top 16 gives about 3-4 more points on GSM8K. This matches the PiSSA paper, which says that training the principal parts learns faster and works better than training the small parts.

Question 4: SVD of q_proj in the original model (layers 0, 7, 15)

	Layer 0	Layer 7	Layer 15
Minimum Target Rank for query projection weights (≥ 50% energy)	62	120	84
σ1 / σ16	31.4 / 5.7	10.3 / 4.2	12.9 / 4.7
Energy in top 16 / top 200	34.8% / 71.2%	15.4% / 63.5%	21.0% / 68.8%







Left: singular value magnitude (bar chart). Right: cumulative energy (line chart). Only the first 200 of 2,048 singular values are shown. Energy = $\Sigma\sigma_i^2 / \Sigma\sigma^2$. The dashed line is 50%.

- **Shape.** All three layers have a similar shape. There are a few large singular values at the beginning (σ1 is about 2.5-5.5 times σ16), and then a long and almost flat tail. To keep 50% of the energy, we only need 62-120 of the 2,048 values (3-6% of the full rank). But even the first 200 values only keep 63-71% of the energy. So the weight is not really low-rank. The rest of the energy is spread over many small values.
- **Differences between layers.** Layer 0 is the most concentrated. σ1 = 31.4 is much bigger than the others, and the top 16 have 34.8% of the energy. Layer 7 is the most spread out (only 15.4% in the top 16, and it needs rank 120). Layer 15 is in the middle (rank 84). My guess is that layer 0 works directly on the token embeddings, so a few main directions carry most of the information. Middle layers mix many different features, so they use more directions. The last layers focus more again on predicting the next token.
- **What this means.** If we use the same rank for every layer, each layer keeps a very different amount of energy. For example, rank 16 keeps about 35% of the energy in layer 0, but only 15% in layer 7. This supports the idea of PiSSA (training the principal part). It also suggests that giving different ranks to different layers may use the parameters better.